High Level Design
on
Mutli – Agent Insurance System

Course Name: AAI

Institution Name: Medicaps University – Datagami Skill Based Course

Student Name(s) & Enrolment Number(s):

Sr no	Student Name	Enrolment Number
1.	YASH SHRIVASTAVA	EN22CS3011111
2.	VISHWAS BHARGAV	EN22CS3011095
3.	YASHI HARNE	EN22CS3011116
4.	DIXIKA VERMA	EN22ME304027
5.	ANVESHA WANKHADE	EN22ME304014

Group Name: Group 07D10

Project Number: AAI - 20

Industry Mentor Name:

University Mentor Name: Prof. Maya Baniya

Academic Year:2025-2026

MEDICAPS UNIVERSITY

Datagami
"Lead Digital Technology"

Table of Contents

1. **Introduction.**
   - 1.1. Scope of the document.
   - 1.2. Intended Audience
   - 1.3. System overview.
2. **System Design.**
   - 2.1. Application Design
   - 2.2. Process Flow.
   - 2.3. Information Flow.
   - 2.4. Components Design
   - 2.5. Key Design Considerations
   - 2.6. API Catalogue.
3. **Data Design.**
   - 3.1. Data Model
   - 3.2. Data Access Mechanism
   - 3.3. Data Retention Policies
   - 3.4. Data Migration
4. **Interfaces**
5. **State and Session Management**
6. **Caching**
7. **Non-Functional Requirements**
   - 7.1. Security Aspects
   - 7.2. Performance Aspects
8. **References**

1. Introduction

The Multi-Agent Insurance System is an AI-powered system that uses multiple intelligent agents working collaboratively to perform complex insurance-related tasks. Instead of relying on a single AI model, this system divides responsibilities among specialized agents that communicate and coordinate with each other.
The system includes:

- Researcher Agent – Responsible for collecting insurance policy information such as coverage, eligibility, and benefits.
- Writer Agent – Responsible for organizing, summarizing, and presenting the gathered information in a structured format.

Using frameworks such as LangChain or CrewAI, the agents communicate with each other, share context, and hand off tasks to ensure high-quality output. The system demonstrates the concept of Agentic AI, where autonomous agents collaborate to solve complex problems.
This approach improves accuracy, modularity, and scalability compared to traditional AI systems.

1.1. Scope of the Document

This document provides a detailed description of the architecture, design, and workflow of the Multi-Agent Insurance System.
It covers:

- System architecture and components
- Agent communication and collaboration
- Process flow and information flow
- Design considerations and system decisions
- API structure used for agent interaction

The document serves as a guide for developers, researchers, and stakeholders to understand how the system operates and how it can be implemented.

1.2. Intended Audience

This document is intended for:

- Insurance professionals seeking clear, visual representations of complex concepts.
- AI developers and data scientists implementing generative AI workflows.
- Educators and trainers designing instructional content.
- Project managers and stakeholders overseeing AI-based innovation initiatives.

It ensures that all readers understand the system's capabilities, relevance, and potential applications.

1.3. System Overview

The Multi-Agent Insurance System consists of multiple AI agents that collaborate to process insurance queries and generate structured responses.

The system workflow is as follows:

1. A user submits an insurance-related query.
2. The Researcher Agent gathers relevant insurance information from knowledge sources or databases.
3. The gathered information is passed to the Writer Agent.
4. The Writer Agent analyzes, summarizes, and formats the information.
5. The system returns a structured response to the user.

Key Features

- Multi-agent collaboration
- Autonomous task delegation
- Context sharing between agents
- Structured content generation
- Scalable architecture

This architecture allows each agent to specialize in a specific task, improving overall system performance.

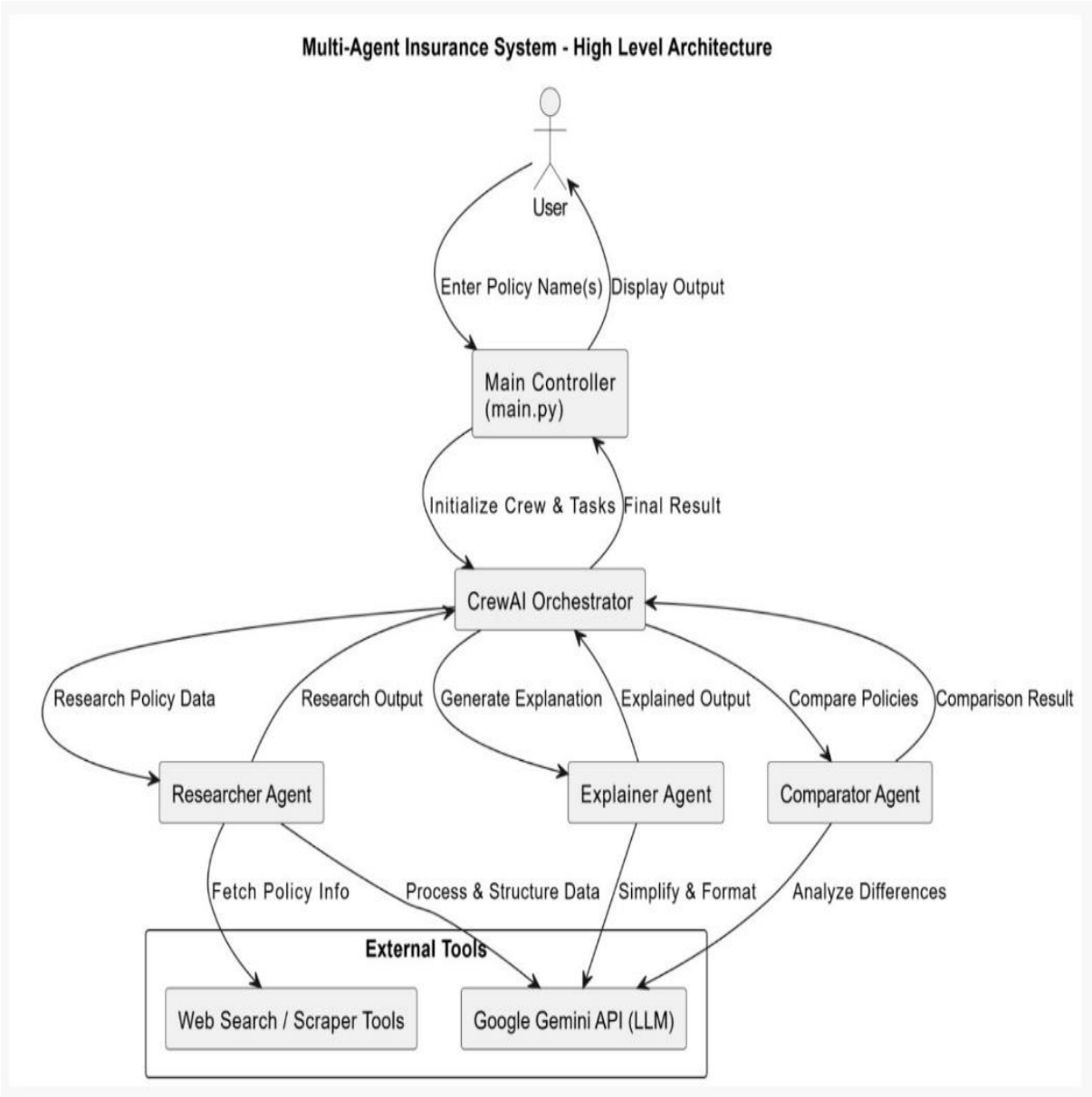
2. System Design.

Architecture Diagram

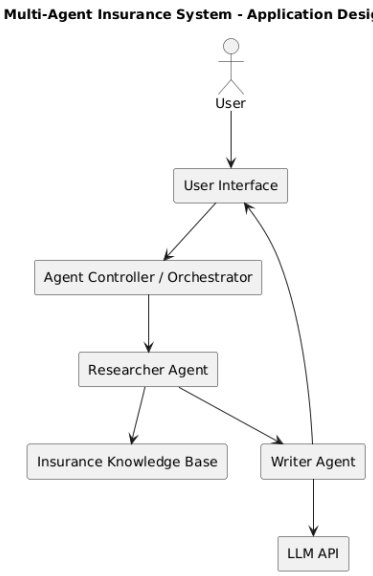
User enters policy name(s) and receives output.

- **Main Controller (main.py)**
  - Takes input
  - Starts the system
  - Displays final result
- **CrewAI Orchestrator** (CrewAI)
  - Manages all agents
  - Assigns tasks
  - Combines results
- **Researcher Agent**
  - Collects policy data
  - Uses web/tools

- **Explainer Agent**
  - Converts data into simple explanation
- **Comparator Agent**
  - Compares multiple policies (if given)
- **External Tools**
  - Web search for data
  - Google Gemini API (using Gemini 2.5 Flash) for processing

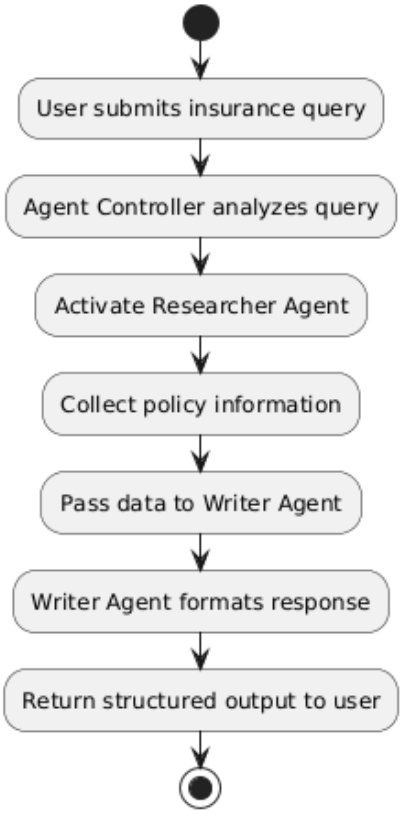


2.1Application Design



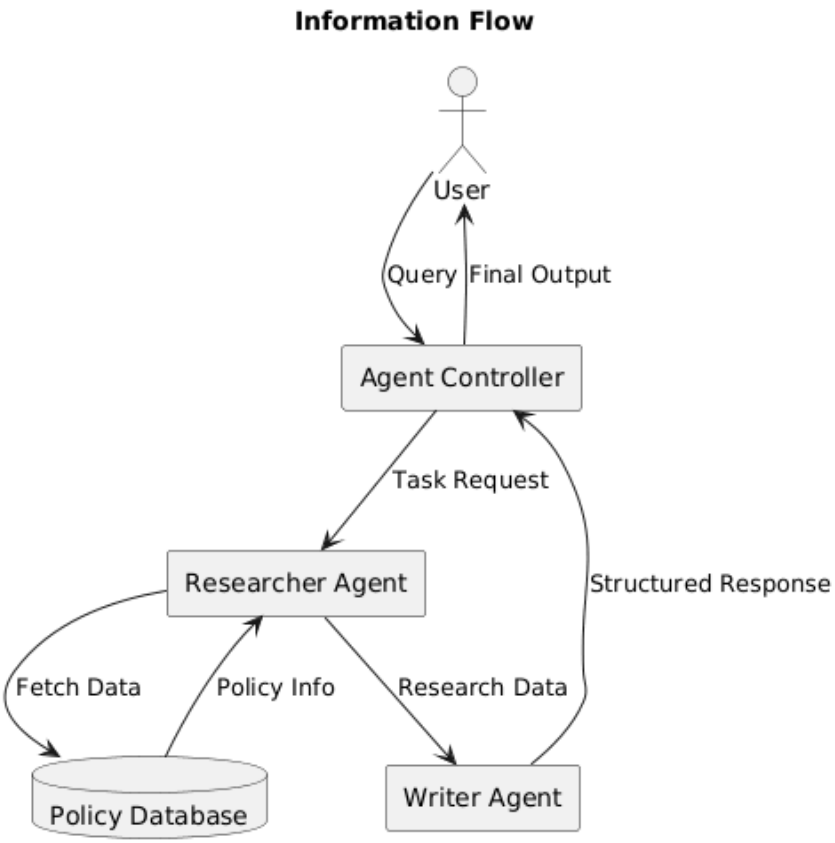
- User submits a query through the User Interface.
- The Agent Controller manages workflow between agents.
- Researcher Agent collects policy data from knowledge sources.
- Data is sent to the Writer Agent.
- Writer Agent uses LLM API to generate structured responses.
- The final response is returned to the user.

2.2Process Flow



- The workflow starts with a user query.
- The system analyzes the request.
- The Researcher Agent retrieves relevant information.
- Information is transferred to the Writer Agent.
- The writer agent organizes and formats the data.
- The system sends the final answer to the user.

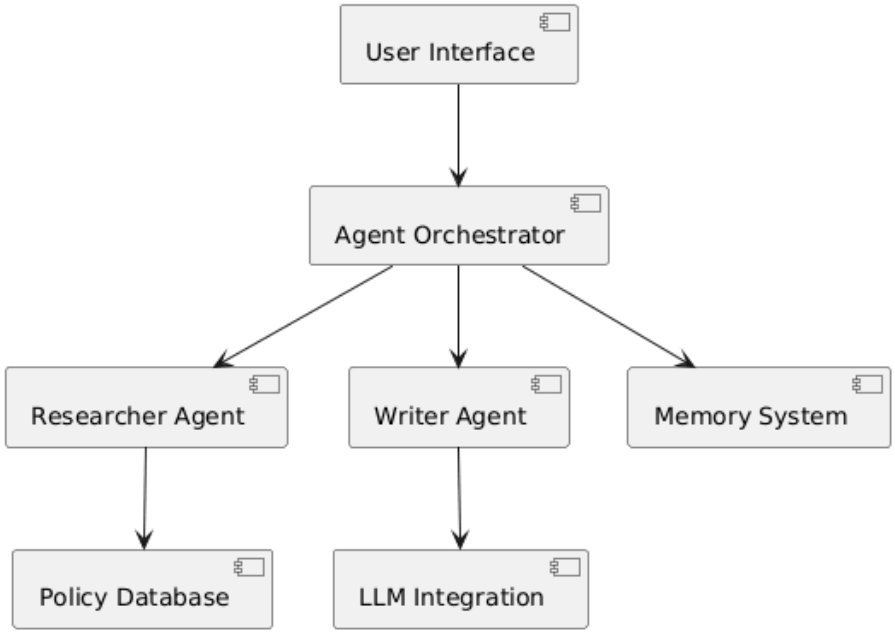
2.3 Information Flow.



- User sends a query to the system.
- The Agent Controller assigns tasks to the Researcher Agent.
- The researcher retrieves policy data from the database.
- Collected data is passed to the Writer Agent.
- Writer Agent creates structured content.
- The system returns the final response to the user.

2.4Components Design

Component Diagram - Multi-Agent Insurance System



- User Interface interacts with users.
- Agent Orchestrator coordinates system activities.
- Researcher Agent collects insurance information.
- Writer Agent generates readable outputs.
- LLM Integration provides language intelligence.
- Memory System stores conversation context.
- Policy Database contains insurance data.

2.5 Key Design Considerations

Several design factors were considered when developing the system.

1. Modularity
Each agent is designed independently, allowing easy upgrades or modifications.

2. Scalability
The architecture allows additional agents such as:
- Risk Analysis Agent
- Policy Recommendation Agent
- Fraud Detection Agent

3. Fault Tolerance
If one agent fails, the system can retry or reassign tasks.

4. Context Management
Maintaining context ensures agents understand previous steps and avoid redundant processing.

5. Security
Sensitive insurance information must be handled securely with proper data protection mechanisms.

6. Performance Optimization
Efficient agent communication reduces processing time and improves response quality.

2.6 API Catalogue:

The API catalogue defines the interfaces used for agent communication.

1. Query Submission API

- **Endpoint**

POST /query

- **Description**

Accepts user queries related to insurance policies.

- **Input**

```
{
  "query": "What does health insurance policy cover?"
}
```

- Output

```
{
  "response": "Health insurance policies generally cover hospitalization, medical treatments, and prescribed medications."
}
```

2. Research Agent API

- **Endpoint**

POST /research

- **Description**

Retrieves insurance policy information.

- **Input**

```
{
  "topic": "health insurance coverage"
}
```

- **Output**

```
{

  "research_data": "Health insurance covers hospitalization, surgery, and medical expenses."

}
```

3. Writer Agent API

- **Endpoint**

POST /write

- **Description**

Formats research data into structured content.

- **Input**

```
{
  "data": "health insurance covers hospitalization..."
}
```

- **Output**

```
{

  "formatted_content": "Health insurance policies provide financial protection against medical expenses..."

}
```

3. Data Design

3.1 Data Model

The Multi-Agent Insurance System employs a hybrid data model integrating structured, semi-structured, and unstructured data to support intelligent agent collaboration.

The core entities include:

- **User Entity**

  Captures user identifiers, metadata, and query inputs.

- **Query Object**

  Represents the lifecycle of a query, including raw input, preprocessed form, intent classification, and contextual enrichment.

- **Agent Artifacts:**
  - *Researcher Agent Output*: Extracted knowledge such as policy clauses, eligibility conditions, and external references.
  - *Writer Agent Output*: Synthesized, human-readable response formatted according to predefined templates.
- **Knowledge Base (KB)**

  Stores insurance-related datasets such as policy documents, claim procedures, underwriting rules, and FAQs. This may include:

  - Structured data (SQL/NoSQL databases)
  - Unstructured data (PDFs, web content)
- **Vector Embeddings Store**

  Enables semantic search by representing documents in high-dimensional vector space.

The system leverages schema-flexible formats (e.g., JSON) to facilitate interoperability across agents and tools.

3.2 Data Access Mechanism

The system employs a tool-augmented, agent-driven data access mechanism:

- **LLM API Access**
  Integration with Google Gemini API enables:
  - Natural language understanding
  - Summarization
  - Content generation
- **CrewAI Task Execution Model**
  - Tasks are assigned to agents with defined roles
  - Researcher Agent retrieves data using tools
  - Writer Agent synthesizes results
- **Retrieval-Augmented Generation (RAG)**
  External knowledge sources are queried before LLM generation to improve factual accuracy.
- **Tool-Based Retrieval**
  - Document loaders

- Vector search (optional integration)
  - Web/API search tools

3.3 Data Retention Policies

The system adheres to privacy-first and minimal retention principles:

- **Ephemeral Session Storage**
  - Context is retained only during active sessions
  - Automatically cleared after session termination
- **Minimal Logging**
  - Logs include anonymized queries and system outputs
  - Used for debugging and system improvement
- **PII Handling**
  - Sensitive data is masked or excluded before storage
- **Retention Window**
  - Logs stored for a limited duration
- **Compliance Alignment**
  Designed to align with global standards such as GDPR principles.

3.4 Data Migration

Data migration is designed for scalability and model evolution:

- **Model Upgrade Compatibility**
  - Supports transition between LLM versions (e.g., Gemini upgrades).
- **Schema Evolution**
  - JSON-based flexible schema allows easy modification.
- **Embedding/Data Migration**
  - If vector databases are used, embeddings can be re-generated when models change.
- **Cloud-Based Storage Migration**
  - Enables portability across cloud platforms.

4. Interfaces

The system provides clearly defined interfaces:

- User Interface (UI)
  - Chat-based interface for query input and response display
- Agent Interface (CrewAI)
  - Defines roles, goals, and tasks for agents
  - Enables structured task delegation and hand-offs
- LLM Interface
  - Interaction with Gemini API for inference
- Tool Interface
  - Allows agents to invoke external tools dynamically

5. State and Session Management

State management is handled using CrewAI's orchestration capabilities:

- Session Initialization
  - Each user interaction is assigned a unique session ID
- Context Management
  - Stores conversation history and intermediate outputs
  - Passed between agents as part of task input
- Agent Memory
  - *Short-Term Memory*: Maintains current workflow context
  - *Task Context Sharing*: Ensures continuity between Researcher and Writer agents
- Workflow State Transitions

User Query → Task Creation → Researcher Agent → Writer Agent → Final Response

- Stateless LLM Handling
  Since Gemini models are stateless, context is explicitly managed and passed in prompts.

6. Caching

Caching improves efficiency and reduces API costs:

- Prompt-Response Cache
  - Stores frequently asked queries and their outputs.
- LLM Output Caching
  - Reduces repeated calls to Gemini API.
- Embedding Cache (if used)
  - Avoids recomputation of embeddings for repeated documents.
- Cache Invalidation
  - Time-based expiration
  - Update-triggered refresh

7. Non-Functional Requirements

7.1 Security Aspects

Security considerations for CrewAI + Gemini-based system include:

- Secure API Usage
  - API keys for Google Gemini API are securely stored (e.g., environment variables)
- Data Encryption
  - HTTPS for all API communications

- Prompt Security
  - Input validation to prevent prompt injection attacks
- Access Control
  - Restrict system usage to authorized users
- Audit Logging
  - Track agent actions and API usage

7.2 Performance Aspects

Performance is optimized for real-time interaction:

- Low Latency Inference
  - Achieved using Gemini 2.5 Flash, which is designed for fast responses.
- Efficient Agent Orchestration
  - CrewAI enables streamlined task execution with minimal overhead.
- Reduced API Calls
  - Through caching and optimized prompt design.
- Scalability
  - Supports concurrent users
  - Can be deployed on scalable cloud infrastructure
- Parallel Task Execution (Optional)
  - Multiple agents can operate concurrently if needed.

8.References

1. Artificial Intelligence: A Modern Approach – by Stuart Russell and Peter Norvig, Pearson Education.
2. Multiagent Systems: Algorithmic, Game-Theoretic, and Logical Foundations – by Yoav Shoham and Kevin Leyton-Brown, Cambridge University Press.
3. LangChain Official Documentation – https://docs.langchain.com
4. CrewAI Official Documentation – https://docs.crewai.com
5. OpenAI API Documentation – https://platform.openai.com/docs